# Unit-II

# WEB SCRIPTING LANGUAGES

**JavaScript:** Introduction to Scripting languages, Introduction to JavaScript (JS), JS Variables and Constants, JS Variable Scopes, JS Data Types, JS Functions, JS Array, JS Object, JS Events. Advanced.
**JavaScript:** JSON - JSON Create, Key-Value Pair, JSON Access, JSON Array, JS Arrow Functions, JS Callback Functions, JS Promises, JS Async-Await Functions, JS Error Handling.
**AJAX:** Why AJAX, Call HTTP Methods Using AJAX, Data Sending, Data Receiving, AJAX Error Handling.
**JQUERY**: Why JQuery, How to Use, DOM Manipulation with JQuery, Dynamic Content Change with JQuery, UI Design Using JQuery.

## Scripting languages

- Scripting languages are types of programming languages where the instructions are written for a run-time environment, to bring new functions to applications, and integrate or communicate complex systems and other programming languages.
- They are widely used in web development, system administration, and automation.
- Ex PHP, Python, JavaScript, and jQuery. Scripting languages are often used for short scripts, and they can automate some code implementation.
- Scripting languages are created for specific runtime environments and use an interpreter to translate commands from source code.

## Advantages of scripting languages:

- **Easy learning:** The user can learn to code in scripting languages quickly, not much knowledge of web technology is required.
- **Fast editing:** It is highly efficient with the limited number of data structures and variables to use.
- **Interactivity:** It helps in adding visualization interfaces and combinations in web pages. Modern web pages demand the use of scripting languages. To create enhanced web pages, fascinated visual description which includes background and foreground colours and so on.

- **Functionality:** There are different libraries which are part of different scripting languages. They help in creating new applications in web browsers and are different from normal programming languages.

**Application of Scripting Languages:** Scripting languages are used in many areas:

- Scripting languages are used in web applications. It is used in server side as well as client side.
- Server-Side scripting languages are: JavaScript, PHP, Perl etc. and client-Side scripting languages are: JavaScript, AJAX, jQuery etc.
- Scripting languages are used in system administration. For example: Shell, Perl, Python scripts etc.
- It is used in Games application and Multimedia.
- It is used to create plugins and extensions for existing applications.

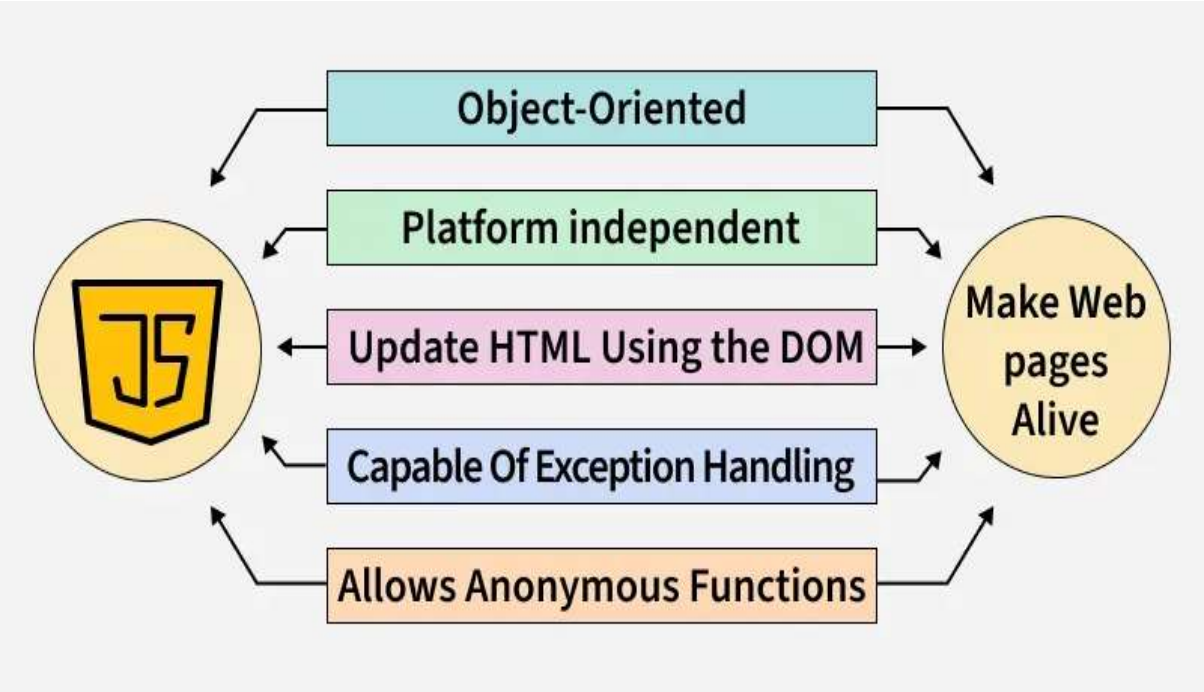
## Introduction to JavaScript (JS)

- Original it is called Live script developed by Netscape in the year of 1995.
- Netscape and sun microsystems jointly called as a JavaScript.
- JavaScript is a versatile, dynamically typed programming language that brings life to web pages by making them interactive.
- It is used for building interactive web applications.
- supports both client-side and server-side development, and integrates seamlessly with HTML, CSS, and a rich standard library.
- JavaScript is single threaded language that executes one task at a time.
- Java script is scripting language, it is light-weight programming language.
- Java script is usually embedded directly into HTML pages.

- JavaScript can be divided into 3 parts

1. Core:- Heart of Language (Operations, expressions, subprogram)
2. Client-side
3. Server-side

Client-side java script is an HTML embedded scripting language; we refer to every collection of java script code as script.




**Run JS Program in different way**

1. **Run JavaScript in Browser**

**Index.html**

```
<!DOCTYPE html>
<html>
<head>
   <title>Run JS</title>
</head>
<body>
<h2 id="demo"></h2>
<script>
   document.getElementById("demo").innerHTML = "Hello World";
</script>
</body>
```

```
</html>
```

**2. Run JavaScript using Browser Console**

```
console.log("Hello World");
```

**3.Run JavaScript using External JS file**

index.html

```
<!DOCTYPE html>
<html>
<body>
<script src="script.js"></script>
</body>
</html>
```

**script.js**

```
document.write("Hello World");
```

**4. Run JavaScript using Node.js (Without browser)**

**Step 1: Install Node.js**

Download from:
https://nodejs.org

**app.js**

```
console.log("Hello World");
```

**Open terminal in folder:**

```
node app.js
```

## JavaScript Variables:

- A variable is a container used to store data that can change during program execution.
- Syntax:
- keyword variableName = value;
- In JavaScript, variables are declared using the keywords var, let, or const.

### 1. var keyword

The var can be re-declared & updated. A Globally-scoped variable.

Ex.

```
var n = 10;
var n = 12; // Re-declaration is allowed
console.log(n);
console.log(n);
```

Output

12

12

### 2. let keyword

The let keyword cannot be re-declared but it can be updated. It has block scope .

Ex

```
let  n = 10;
     n = 20; // Value can be updated
// let n = 15; //can not redeclare
console.log(n)
```

Output

20

### 3. const keyword

const keyword -variable can not be re-declared & can not be updated. A block-Scope variable.

**Ex**

```
const PI = 3.14;
// PI = 200; This will throw an error
    console.log(PI);
```

**Output**

3.14

## Variables Rules

- Variables names are case sensitive.
- Only letter, digits and underscore(_) & $ is allowed.(not even space).
- Only letter ,Underscore & $ should be first character.
- Reserved name can not be variable name.

## Data Types in JS

JavaScript supports various datatypes, which can be broadly categorized into primitive and non-primitive types.

Primitive Datatypes

Primitive datatypes represent single values and are immutable.

1. Number: Represents numeric values (integers and decimals).

```
let n=42;
let pi=3.14;
```

2. String: Represents text enclosed in single or double quotes

```
let s="Hello World !!!";
```

3. Boolean: Represents a logical value (true or false).

```
let  bool=true;
```

**4.Undefined:** A variable that has been declared but not assigned a value.

**Ex** let x;

console.log(x);

5.**Null** :Special type of variable used to specifies NULL value.

**Ex**

let x=null;

6.**BigInt:** It used for storing big integers.

**Ex**

let x=BigInt("123");

7.**Symbol:**

let y=Symbol("Hello");

**Non-Primitive Datatypes**

Non-primitive data types are used to store multiple values or complex data.

**1. Object:** Represents key-value pairs.

**Ex** const student = {

fullName : "Amit Sharma",

age : 25,

cgpa : 9,

isPass :true

};

To Access particular value we can access by using object name & key

**obj.key or obj ["key"]**

**Ex**

student.age or student["age"]

## JS Array

- An array stores multiple values in a single variable.
- Used for lists or collections

**Syntax:**

let arr = [value1, value2, value3];

Ex

let fruits = ["Apple", "Banana", "Mango"];

console.log(fruits[0]);

**Array Methods**

**1. push() → add element**

fruits.push("Orange");

**2. pop() → remove last**

fruits.pop();

**3. shift() → remove first**

fruits.shift();

**4. unshift() → add first**

fruits.unshift("Grapes");

**5. length**

console.log(fruits.length);

## JavaScript Functions

- Functions in JavaScript are reusable blocks of code designed to perform specific tasks.
- They allow you to organize, reuse, and modularize code. It can take inputs, perform actions, and return outputs.
- function keyword is used to defined functions in JS.

**Ex**

```
function sum(x, y)
{
   return x + y;
}
console.log(sum(6, 9));
```

**Output**
15

## Callback Functions

- A callback function is passed as an argument to another function and is executed after the completion of that function.

**Ex**

```
function greet(name, callback)
{
   console.log("Hello " + name);
   callback();
}
```

## Anonymous Function

- Anonymous functions are functions without a name. They are often used as arguments to other functions
- Ex.

```
 const add= function(a, b)
 {
  return a+b;
 }
```

**Ex Anonymous Function as a call back**

```
setTimeout(function() {
   console.log("After 2 seconds");
}, 2000);
```

## Arrow Function

- **Arrow Functions** allow a shorter syntax for **function expressions**.
- You can skip the **function** keyword, the **return** keyword, and the **curly brackets**

- **Before Arrow**

```
function add(a, b)
{
  return a + b;
}
```

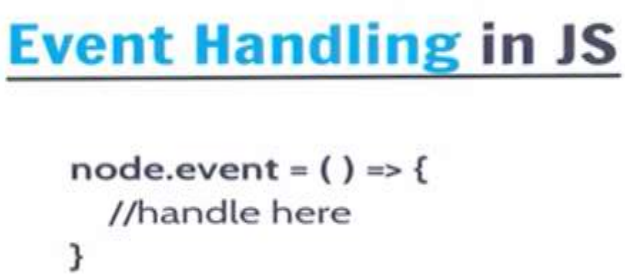
**With Arrow Function**

```
const add = (a, b) => a + b;
```

## JavaScript Events

- The change in the state of an object is known as an Event. In html, there are various events which represents that some activity is performed by the user or by the browser.
- When javascript code is included in HTML, js react over these events and allow the execution. This process of reacting over the events is called Event Handling. Thus, js handles the HTML events via Event Handlers.
- For example

- Mouse events (click, double click etc.)
- Keyboard events (keypress, keyup, keydown)
- Form events (submit etc.)
- Print event & many more




**First.js**

```
let btn1=document.querySelector("#btn1");
```

```
btn1.onclick=()=>
{
  console.log("btn1 was click");
};
let div=document.querySelector("div");
div.onmouseover=()=>
{
  console.log("You are inside div");
}
```

**Index.html**

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Event in JS</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <button id="btn1">btn1</button>
   <div>
    this is box
</div>
   <script src="first.js"></script>

</body>
</html>
```

**Style.css**

```
div{
    height:100 px;
    width:100 px;
    background-color:aqua;
```

```
    color: white;
    border: 1px solid black;
}
```

## Mouse events

| Event Performed | Event Handler | Description |
|---|---|---|
| click | onclick | When mouse click on an element |
| mouseover | onmouseover | When the cursor of the mouse comes over the element |
| mouseout | onmouseout | When the cursor of the mouse leaves an element |
| mousedown | onmousedown | When the mouse button is pressed over the element |
| mouseup | onmouseup | When the mouse button is released over the element |
| mousemove | onmousemove | When the mouse movement takes place |

## Keyboard events:

| Event Performed | Event Handler | Description |
|---|---|---|
| Keydown & Keyup | onkeydown & onkeyup | When the user press and then release the key |

## Form events:

| Event Performed | Event Handler | Description |
|---|---|---|
| focus | onfocus | When the user focuses on an element |
| submit | onsubmit | When the user submits the form |
| blur | onblur | When the focus is away from a form element |
| change | onchange | When the user modifies or changes the value of a form element |

## Window/Document events

| Event Performed | Event Handler | Description |
|---|---|---|
| load | onload | When the browser finishes the loading of the page |
| unload | onunload | When the visitor leaves the current webpage, the browser unloads it |
| resize | onresize | When the visitor resizes the window of the browser |
| reset | onreset | When the window size is resized |
| scroll | onscroll | When the visitor scrolls a scrollable area |

## Advanced JavaScript

What is JSON?

- JSON stands for JavaScript Object Notation.
- JSON is a lightweight data interchange format.
- JSON is language independent.
- JSON is "self-describing" and easy to understand.
- JSON is a format for storing and transporting data.
- JSON is often used when data is sent from a server to a web page.

## JSON Rules

- Data in key–value pairs
- Keys must be in double quotes
- Values can be:

string

number

boolean

array

object

null

**Ex**

```
let user = {
  "name": “John",
  "age": 22,
  "city": "Pune"
};
```

**JSON Access**

**1. Dot notation**

```
console.log(user.name);
```

**2 Bracket notation**

```
console.log(user[“name"]);
```

## JSON Array

A **JSON Array** is a collection of **multiple values stored in order**.

- It is used to store:
- list of students
- products
- Employees
- API data
- multiple objects

**Ex**

```
let students = [
  {"name": “Amit", "age": 20},
  {"name": “Rahul", "age": 21}
];
```

## Convert JSON in JavaScript

**1. Object → JSON string**

***JSON.stringify(user);***

**2. JSON string → Object**

JSON.parse(jsonString);

**1. Object → JSON String**

JSON.stringify() : It Converts JavaScript Object into JSON formatted string.

It Used when:

- Sending data to server
- Saving data in file
- Storing in localStorage
- API requests

**Syntax**

JSON.stringify(object, replacer, space)

**Example**

```
let user = {
  name: "Shraddha",
  age: 22,
  city: "Pune"
};
let jsonString = JSON.stringify(user);
console.log(jsonString);
```

**Output**

```
{"name":"Shraddha","age":22,"city":"Pune"}
```

**Example with Array**

```
let students = [
  {name: "A", age: 20},
  {name: "B", age: 21}
];
let str = JSON.stringify(students);
console.log(str);
```

**Output**

[{"name":"A","age":20},{"name":"B","age":21}]

**2. JSON String → Object**: It Converts JSON string into JavaScript Object.

It Used when:

- Getting API response
- Reading stored data
- Fetch requests

**Syntax**

JSON.parse(jsonString)

**Example**

```
let str = '{"name":"Shraddha","age":22,"city":"Pune"}';

let obj = JSON.parse(str);

console.log(obj);
```

**Output**

{ name: "Shraddha", age: 22, city: "Pune" }

**Array Example**

```
let data = '[{"name":"A"},{"name":"B"}]';

let arr = JSON.parse(data);

console.log(arr[0].name);
```

**Output**

A

**Synchronous**

- Synchronous means the code runs in a particular sequence of instructions given in the program.
- **Ex.**

```
console.log("One");
console.log("Two");
console.log("Three");
```

**Output**

One
Two
Three

**Asynchronous**

- Due to synchronous programming, sometimes imp instructions get blocked due to some previous instructions, which causes a delay in the program execution.
- Asynchronous code execution allows to execute next instructions immediately and doesn't block the flow.

**Ex**

```
console.log("One");
console.log("Two");
   setTimeout(() =>
{
  console.log("Hello!!!!");
}, 2000);
console.log("Three");
console.log("Four");
```

**Output**

One

Two

Three

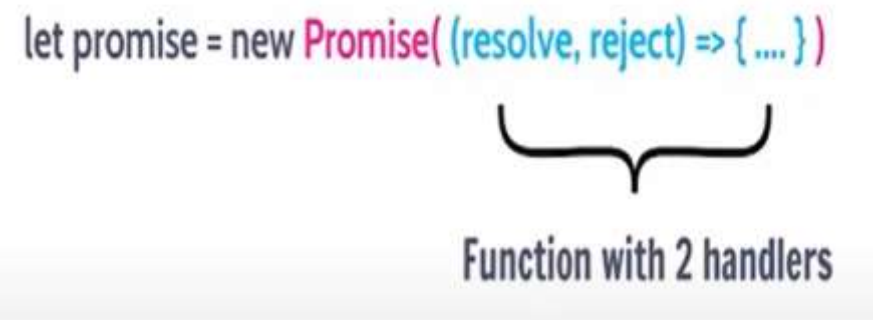
Four

Hello!!!!

**JS Promises**

- Promises is used for **eventual** completion of task.
- It is an Object in JS.

➢ It is solution to call back hell.




Promise have 3 state

**1.Pending**

Operation still running

```
let promise=new Promise((resolve,reject)=> {
  console.log("Hii I am promise");
});
```

**2. Fulfilled**

Success → result returned

```
let promise=new Promise((resolve,reject)=>{
  console.log("hii i am promise")
  resolve("success");
});
```

**3. Rejected**

Error → failure returned

```
let promise=new Promise((resolve,reject)=>{
  console.log("hii i am promise")
  reject("Some error occured");
});
```

```
        let promise=new Promise((resolve,reject)=>{
      console.log("hii i am promise");
```

```
});
```

**Method of Promise**

1) **promise.then ( ( res ) => {…..})**
2) **promise.catch ( (err ) => {…..})**

**Ex**

```
const getPromise = () => {
return new Promise((resolve, reject) => {
  let number = 4;
  if (number % 2 == 0) {
    resolve("The number is even!");
  } else {
    reject("The number is odd!");
  }  });
};
let promise = getPromise();
promise.then((res) => {
  console.log("Promise fulfilled:", res);
})
promise.catch((err) => {
  console.log("Promise rejected:", err);
});
```

**JS Async-Await Functions**

**async**

- Makes a function **asynchronous**
- Always returns a **Promise**

**await**

- Used **inside async function only**
- Waits for a **Promise to resolve**
- Pauses execution until result comes

**Syntax**

**Syntax :**

```
async function functionName() {

let result = await promise;
}
```

**Ex**

```
function delay() {
   return new Promise(resolve =>
      setTimeout(resolve, 2000)
   );
}
async function test() {
      console.log("1 Hello");
      console.log("2:I");
      await delay();   // waits 2 seconds
       console.log("3:am");
      console.log("4:in");
}
test();
console.log("5:Sangamner");
```

**When to use async/await?**

Use when:

- API calls
- Database calls

- File reading
- Timers
- Any async task

## JS Error Handling

- JavaScript error handling is a crucial mechanism for managing unexpected issues during code execution, allowing programs to recover gracefully instead of crashing.
- The primary constructs for this are the try...catch...finally statements and the throw statement.

**Mechanism use to handle error**

**try...catch**: This block is used to enclose code that might throw an error.

- The try block contains the code to run.
- If an exception occurs within the try block, control immediately shifts to the catch block.
- The catch block receives an error object, which provides information about the error (e.g., name, message, stack trace)

```
try {
  // Code that may potentially throw an error
  someFunctionThatMightFail();
} catch (error) {
  // Code to handle the error
  console.error("An error occurred:", error.message); // Use console.error for debugging
}
```

**finally**: This block contains code that will *always* execute after the try and catch blocks finish, regardless of whether an error occurred or was caught. It is ideal for cleanup operations like closing files or hiding loading spinner

```
try {
  // ... code that might throw ...
} catch (error) {
```

```
  // ... handle error ...
} finally {
  // Code that always runs (e.g., resource cleanup)
  console.log("Execution finished.");
}
```

**throw**: The throw statement allows you to create and manually raise a custom exception. You can throw any JavaScript object, but it's best practice to throw an instance of the built-in Error object or a custom error class.

```
const age = 16;
if (age < 18) {
  throw new Error("Age must be 18 or above.");
}
```

**Error Types**

JavaScript has several built-in Error types:

- **SyntaxError**: Thrown when there's an issue with the code structure (e.g., missing parentheses).

```
console.log("Hello World"
// Missing closing parenthesis
```

- **ReferenceError**: Occurs when trying to access a non-existent variable or function.

```
console.log(x); // ReferenceError: x is not defined
```

- **TypeError**: Thrown when an operation is performed on a value of an incompatible type (e.g., calling a method on null or undefined).

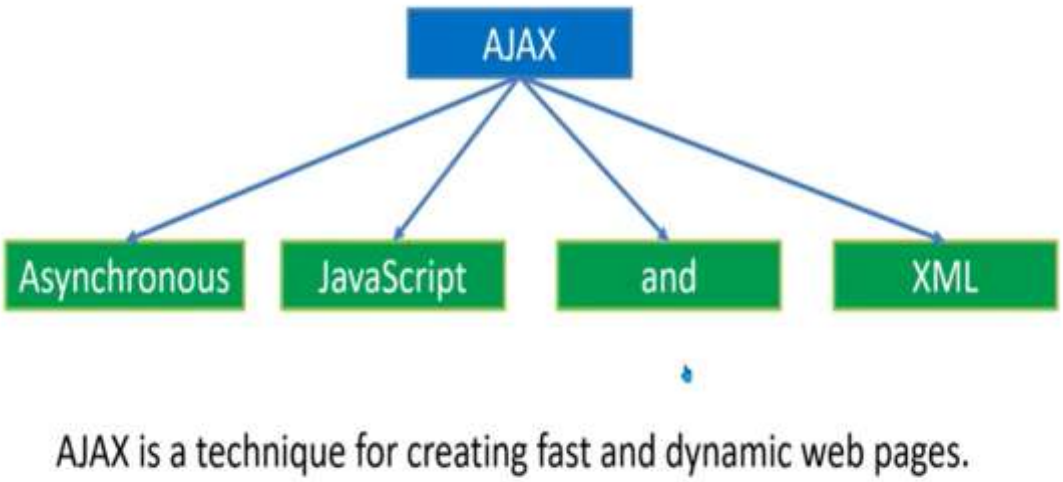
```
let num = 5;
num.toUpperCase(); // TypeError: num.toUpperCase is not a function
```

- **RangeError**: Occurs when a numeric value is outside the range of allowed values.

```
let arr = Array(-1); // RangeError: Invalid array length
```

- **URIError**: Thrown when invalid arguments are passed to URI handling functions like decodeURI()

**AJAX**




AJAX stands for:

- Asynchronous JavaScript And XML
- It is a web development technique used to update web pages without reloading the entire page.
- AJAX allows JavaScript to send and receive data from the server in the background (asynchronously) and update only part of the webpage.

**Why AJAX is used?**

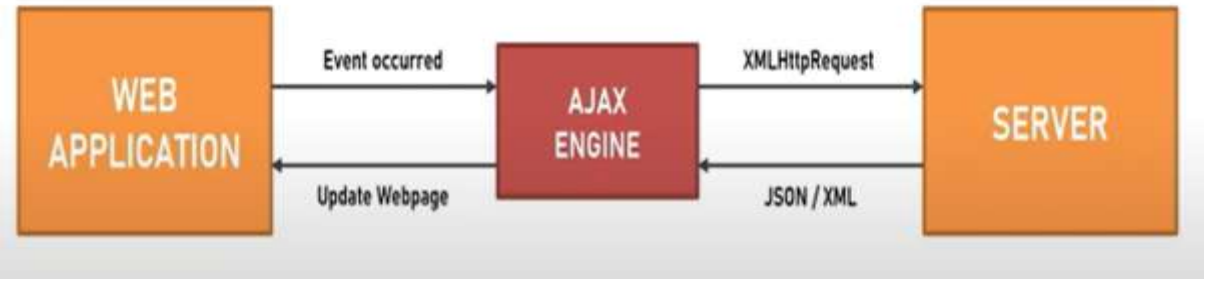
Without AJAX:

- Every request → Page reload

With AJAX:

- Data loads in background
- Only required part updates
- Faster + smoother UI

**How AJAX is work**




**Working**

**Step 1: User Sends Request**

- The user clicks a button / submits a form / types something.
- This action triggers a JavaScript function.

**Step 2: JavaScript Creates AJAX Request**

- JavaScript creates an **XMLHttpRequest object**
- It sends a request to the server in the background
- The page does NOT reload

**Step 3: Request Sent to Server**

The AJAX engine sends the request to the web server.

Server receives:

- URL
- Method (GET/POST)
- Data (if any)

**Step 4: Server Processes Request**

The server:

- Executes backend code (PHP/Node.js/Java/Python etc.)
- May fetch data from database
- Prepares response (JSON/XML/Text)

**Step 5: Server Sends Response**

Server sends data back to browser.

**Step 6: AJAX Receives Response**

JavaScript waits for response using:

**Step 7: Page Updates Dynamically**

Now:

- Only specific part of webpage updates
- Entire page does NOT reload

**ReadyState**

It holds the status of the XMLHttpRequest.

0 : XMLHttpRequest object has been created but request not initialized
1: Server connection established
2 : Request Received
3 : Processing request
4 : Request finished & response is ready

**status**

➢ It returns HTTP response status-code

| Value | Meaning |
|---|---|
| 0 | Before the request completes or XMLHttpRequest show error ,the status code is 0. |
| 200 | OK |
| 403 | Forbidden |
| 404 | Page not found |

**Basic syntax of XMLHttpRequest**

```
var xhr = new XMLHttpRequest();
xhr.open("METHOD", "URL", true);
xhr.onload = function()
{
  console.log(xhr.responseText);
};
```

xhr.send();

## XMLHttpRequest Object Method

1) Open (method ,URL, async)

Method : GET, POST, PUT, DELETE

URL : www.example.com/bookings , newfile.js

Async : true/false

Ex

1) let xhr = new XMLHttpRequest();
   xhr.open(“ GET ”, “ newfile.js ”,true);
2) let xhr = new XMLHttpRequest();
   xhr.open(“ POST ”, “ www.example.com/bookings ”, true);

**2) Send () /send (string)**

- Example for “ GET ” Method :

```
let xhr = new XMLHttpRequest();
xhr.open(“ GET ”, “ newfile.js?id=24 ”, true);
xhr.send();
```

- Example for “ POST ” Method :

```
let xhr = new XMLHttpRequest();
xhr.open(“ POST ”, “ www.example.com/bookings ”, true);
xhr.setRequestHeader(“ Content-type ” , “application/x-www-form-urlencoded”);
xhr.send(“pass=24”);
```

## Use & Importance of AJAX

- AJAX enables web applications to request and receive data from a server in the background, without interrupting the user's interaction with the page.
- It allows specific sections of a web page to be updated dynamically, without requiring a full-page refresh.
- By updating content smoothly and quickly, AJAX contributes to a more responsive and engaging user experience.
- AJAX facilitates real-time data validation, allowing users to receive immediate feedback on their input without submitting the entire form.
- Examples : Google Maps, Gmail, Facebook, and Twitter are popular examples of web applications that heavily rely on AJAX for their functionality.

**Real Life Examples**

You use AJAX when:

- Gmail loads new emails without refresh
- Instagram likes update instantly
- Search suggestions appear while typing
- Form submits without page reload

**Advantages of AJAX**

- Faster websites
- No page reload
- Better user experience
- Less server load
- Smooth UI

**Disadvantages**

- More JavaScript complexity
- Browser history issues
- SEO challenges sometimes

**jQuery**

jQuery is a fast, lightweight JavaScript library that simplifies common web development tasks, such as HTML DOM manipulation, event handling, animation, and Ajax interactions, with its "write less, do more" philosophy.

**Why Use jQuery**

jQuery abstracts away many complexities of raw JavaScript, primarily:

- **Simplified Syntax:** It condenses many lines of JavaScript code into single, concise methods.
- **Cross-Browser Compatibility:** It automatically handles browser inconsistencies, ensuring your code works reliably across different browsers.
- **DOM Manipulation & Traversal:** It provides an easy-to-use API for navigating the Document Object Model (DOM) and selecting elements using familiar CSS-like selectors (e.g., $("#id"), $(".class"), $("tag")).
- **Rich Features & Plugins:** It offers built-in methods for events, effects, animations, and a vast ecosystem of community-created plugins for enhanced functionality.

**DOM Manipulation with jQuery**

1.Selecting Elements

- In JQuery, selecting elements is similar to CSS. JQuery selection allows you to target and manipulate HTML elements on a web page. The basic syntax for selecting elements in JQuery is similar to CSS selectors.

**Syntax:**

- $("element_name_id_class")

Ex.

$("#id") // select by id

$(".class") // select by class

$("p") // select by tag

2. Changing Content

In JQuery, manipulating elements involves selecting HTML elements and performing various actions like changing their content, modifiying attributes, applying styles, or handling events.

- **Chaning Text Content:** To change the text content of an element, you can use the 'text()' method.

**html()**

- Changing HTML Content: To change the HTML content of an element, use the HTML () method.

**Example**

```
<!DOCTYPE html>
<html>
<head>
   <title>jQuery text() and html() Example</title>

   <!-- jQuery Google CDN -->
   <script
   src="https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery.min.js"></scrip
   t>
</head>
<body>
   <p>This is old paragraph text.</p>
   <div id="myDiv">Old Div Content</div>
   <script>
      $(document).ready(function() {
         // text() example
         $('p').text('Hello World From India');

         // html() example
        //  $('#myDiv').html('<strong>New HTML content</strong>');

      });
   </script>

</body>
</html>
```

**3. Changing CSS**

```
<!DOCTYPE html>
<html>
<head>
   <title>jQuery CSS Example</title>
```

```
    <!-- Google CDN -->
    <script
    src="https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery.min.js"></scrip
    t>
</head>
<body>

    <p id="demo">This is a paragraph.</p>

    <button id="btn1">Change Color</button>
    <button id="btn2">Change Multiple CSS</button>

    <script>
        $(document).ready(function(){

            // Single CSS property
            $("#btn1").click(function(){
                $("#demo").css("color", "blue");
            });

            // Multiple CSS properties
            $("#btn2").click(function(){
                $("#demo").css({
                    "color": "red",
                    "font-size": "20px",
                    "background-color": "yellow"
                });
            });

        });
    </script>

</body>
</html>
```

**4. Adding / Removing Elements**

**append()**

```
$("#box").append("<p>New Paragraph</p>");
```

**prepend()**

```
$("#box").prepend("<p>Start Paragraph</p>");
```

**remove()**

```
$("#demo").remove();
```

**Example**

```
<!DOCTYPE html>
<html>
<head>
   <title>jQuery append, prepend, remove Example</title>

   <!-- jQuery CDN -->
   <script
   src="https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery.min.js"></scrip
   t>
</head>
<body>

   <div id="box" style="border:1px solid black; padding:10px;">
      <p id="demo">Original Paragraph</p>
   </div>
   <br>
   <button id="btn1">Append</button>
   <button id="btn2">Prepend</button>
   <button id="btn3">Remove</button>
   <script>
      $(document).ready(function(){

         // append()
         $("#btn1").click(function(){
            $("#box").append("<p>New Paragraph (Added at End)</p>");
         });

         // prepend()
         $("#btn2").click(function(){
            $("#box").prepend("<p>New Paragraph (Added at Start)</p>");
         });

         // remove()
         $("#btn3").click(function(){
            $("#demo").remove();
```

```
        });

    });
  </script>

</body>
</html>
```

**5. Handling Events**

In JQuery, Handling events is very easy. It is also a crucial aspect of web development. Handling events allows you to respond to user interaction such as clicks, keypresses, or form submission.

Here's a basic example of Handling Events in JQuery

**Example**

```
<!DOCTYPE html>
<html>
<head>
  <title>jQuery Example</title>
  <!-- jQuery CDN -->
  <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery.min.js"></script>
</head>
<body>
  <p>This is old text</p>
  <button>Click Me</button>

  <script>
    $(document).ready(function() {

      // Selecting button element
      $('button').click(function() {
```

```
            $('p').text("Hello World from India");
        });
    });
  </script>
</body>
</html>
```

Other events:

- mouseover()
- mouseout()
- keyup()
- change()

**Dynamic Content Change**

Dynamic content means:

➢ Changing webpage content
➢ Without refreshing the page
➢ Based on user actions (click, input, hover, etc.)

This is one of the biggest advantages of jQuery.

**1.Changing Text Dynamically**

```
<!DOCTYPE html>
<html>
<head>
  <title>Dynamic Text Change</title>
  <script
  src="https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery.min.js"></scrip
  t>
</head>
<body>
<p id="demo">Original Text</p>
```

```
<button id="btn">Change Text</button>


<script>
$(function(){
   $("#btn").click(function(){
      $("#demo").text("Text Changed Dynamically!");
   });
});
</script>
</body>
</html>
```

**2. Changing HTML Content Dynamically**

```
$("#demo").html("<b>Bold Dynamic Text</b>");
```

**3.Show / Hide Content Dynamically**

```
$("#demo").hide();
$("#demo").show();
$("#demo").toggle();
```

**4.Animation-Based Dynamic Changes**

```
$("#box").fadeIn();
$("#box").fadeOut();
$("#box").slideToggle();
```

**5.Dynamic Content from User Input**

**Example: Display What User Types**

```
<input type="text" id="name">
<button id="show">Show Name</button>
<p id="result"></p>
```

```
<script>
$(function(){
   $("#show").click(function(){
      let userName = $("#name").val();
      $("#result").text("Hello " + userName);
   });
});
</script>
```

**6.Loading External Content (AJAX)**

```
$("#loadBtn").click(function(){
   $("#content").load("data.txt");
});
```

✓ Loads external file
✓ No page refresh

## UI Design Using JQuery

UI design using jQuery is primarily accomplished through jQuery UI, a library that provides a curated set of interactive widgets, effects, and themes built on top of the core jQuery JavaScript library.

This frontend library allows developers to add rich, desktop-application-like experiences to web applications

### Key Features for UI Design

jQuery UI enhances user interfaces through four main categories:

- **Interactions:** Adds basic mouse behaviors like Draggable, Droppable, Resizable, Selectable, and Sortable.
- **Widgets:** Provides full-featured UI controls such as Accordion, Autocomplete, Datepicker, Dialog, Slider, and Tabs.
- **Effects:** Supports animations for colors, class transitions, and custom visual effects, including options like bounce, explode, highlight, puff, shake, and slide.

- **Utilities:** Offers helper functions for tasks like element positioning (Position) and generating unique IDs (uniqueId)

**Theming and Styling**

jQuery UI utilizes a common styling framework for a consistent appearance.

- **ThemeRoller:** A web application on the jQuery UI website for designing and downloading custom themes. Users can choose from existing themes or create their own.
- **CSS Framework:** Themes use standardized CSS classes for compatibility with the ThemeRoller system.

**1. Changing Styles Dynamically (Interactive UI)**

You can design UI by changing CSS dynamically.

**Example: Theme Change Button**

```
<!DOCTYPE html>
<html>
<head>
  <title>UI Design Example</title>
  <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery.min.js"></script>
</head>
<body>
<h2 id="title">Welcome to My Website</h2>
<button id="darkMode">Dark Mode</button>
<script>
$(function(){
  $("#darkMode").click(function(){
    $("body").css({
      "background-color": "black",
```

```
            "color": "white"
        });
    });
});
</script>

</body>
</html>
```

✓ Changes entire page theme
✓ Creates dark mode UI

2.**Hover Effects (Modern Button Design)**

```
$("#btn").hover(
    function(){
        $(this).css("background-color", "green");
    },
    function(){
        $(this).css("background-color", "blue");
    }
);
```

✓ Used in buttons
✓ Used in menus

**3.Show / Hide Effects (Dropdown UI)**

```
$("#menuBtn").click(function(){
    $("#menu").slideToggle();
});
```

✓ Used in navigation menus
✓ Used in FAQ sections

4. **Animations for Attractive UI**

```
$("#box").animate({
   left: "250px",
   opacity: 0.5
});
```

✓ Used in banners
✓ Used in cards
✓ Used in dashboards

5 **Form UI Enhancement**

```
$("input").focus(function(){
   $(this).css("border", "2px solid blue");
});

$("input").blur(function(){
   $(this).css("border", "1px solid gray");
});
```

✓ Highlights input fields
✓ Improves user experience

**6.Tabs, Accordion, Datepicker (Advanced UI)**

For advanced components, we use:

jQuery UI

It provides ready-made UI components like:

- Accordion
- Tabs
- Datepicker
- Draggable
- Droppable

- Dialog box

**Example: Simple Accordion (jQuery UI)**

```
<div id="accordion">
  <h3>Section 1</h3>
  <div><p>Content 1</p></div>
  <h3>Section 2</h3>
  <div><p>Content 2</p></div>
</div>
<script>
$("#accordion").accordion();
</script>
```

✓ Professional UI
✓ Minimal code